

Pré ING1-MEF1 groupe H

HAMDOUNI Fares, TAKHARBOUCHT Aya, TA DAC Arno

Rapport de Projet C : Cavity-Taskforce



Introduction

Le projet "Cavity-Taskforce" est une simulation de gestion de cabinet dentaire développée en langage C. L'objectif était de créer un jeu en console avec des mécaniques de gameplay variées : gestion du temps, système d'hygiène, inventaire et patients. Le code a été organisé en plusieurs fichiers dédiés (affichage, patients, sauvegarde, objets) pour rester modulaire et lisible.

1. Cadrage et choix technologiques (TD1 — 16/04)

Lors du premier TD nous nous sommes mis d'accord sur le sujet, relu le cahier des charges et établi un plan de travail avec un schéma des liens entre les différentes structures.

Très vite on a été confrontés à un premier problème : tout ce qu'on avait vu en TD utilisait `scanf`, qui bloque le programme en attendant une réponse de l'utilisateur, ce qui ne correspondait pas du tout à l'idée d'un jeu en temps réel. On a eu deux options : utiliser la SDL3 ou rester en terminal avec `getch`. On a essayé la SDL3 pendant deux jours mais c'était trop complexe à maîtriser en même temps que les bases du C. On a donc choisi de tout faire en terminal.

2. Structures et portabilité (TD2 — 07/05)

D'ici la fin du deuxième TD on avait une grille de cases, un dentiste qui ne se déplaçait pas encore et des structures pas encore reliées entre elles (le personnage et les cellules pour récupérer les objets).

Un autre problème est apparu : on codait sur des systèmes différents (Windows et Mac) et les fonctions de capture de touches au clavier ne sont pas les mêmes selon le système.

d'exploitation. On a résolu ça avec des directives de préprocesseur (`#ifdef _WIN32`) pour adapter automatiquement le code selon le système, ce qui nous a quand même pas mal ralenti.

La gestion des conflits Git a aussi été une source de difficultés, notamment quand plusieurs membres modifiaient les mêmes fichiers en même temps.

3. Moteur de jeu et interface (TD3 — 12/05)

Au troisième TD on avait une grille avec un personnage qui pouvait se déplacer et ramasser des objets affichés dans un cadre sur le côté.

On a ensuite finalisé la boucle de jeu avec le système de soins : le joueur doit utiliser le bon outil selon la maladie générée aléatoirement pour chaque patient. On a aussi implémenté le système de plateaux stériles/sales, la gestion de la contamination du matériel et la barre de patience des patients. L'interface a été enrichie avec des couleurs ANSI pour afficher en temps réel les scores, le prix des outils et l'état de la salle d'attente.

Répartition du travail

Le travail s'est organisé de manière flexible, sans planning fixe, avec des sessions généralement en fin de semaine. Chaque membre s'occupait d'une partie du code selon ses avancées :

- Gestion des patients et système de soins (`patients.c`, `patients.h`)
- Affichage et boucle de jeu (`display.c`, `display.h`)
- Sauvegarde et menu principal (`save.c`, `main.c`)

Ce qu'il manque / ce qu'on a fait en plus

Nous avons réussi à développer un moteur de jeu fonctionnel intégrant la logique complète des soins dentaires, la gestion stricte de l'hygiène, une économie interne et des affichages dynamiques comme le suivi des symptômes et l'écran de Game Over. En

revanche, faute de temps, nous n'avons pas pu implémenter le mode coopératif à deux joueurs, le menu de démarrage interactif, ni le classement final.

Conclusion

Ce projet nous a permis d'approfondir nos compétences en C, de la manipulation de structures complexes à la compilation multi-fichiers, en passant par la portabilité du code et le travail collaboratif avec Git.